UCME: Unified and Configurable Matrix Extension for Diverse CPU Architectures with Minimal Design Overhead

Anonymous Author(s)

Abstract

Matrix extensions have emerged as an essential feature in modern CPUs to address the surging demands of AI workloads. However, existing designs often incur substantial hardware and software design overhead. Tight coupling with the CPU pipeline units complicates integration across diverse CPUs, while fine-grained synchronous instructions hinder the development of high-performance kernels.

This paper proposes a unified and configurable CPU matrix extension architecture. By decoupling matrix units from the CPU pipeline, the design enables low-overhead integration while maintaining close coordination with existing compute and memory resources. The configurable matrix unit supports mixed-precision operations and adapts to diverse compute demands and memory bandwidth constraints. An asynchronous matrix multiplication abstraction with flexible granularity conceals hardware details, simplifies matrix-vector overlap, and supports a unified software stack.

The architecture is integrated into four open-source CPU RTL platforms and evaluated on representative AI models. Matrix unit utilization under GEMM workloads exceeds 90% across all platforms. When configured with compute throughput and memory bandwidth comparable to Intel AMX, our design achieves speedups of 1.57×, 1.57×, and 2.31× on ResNet, BERT, and Llama3, with over 30% of the gains attributed to overlapped matrix-vector execution. A 4 TOPS@2GHz matrix unit occupies only 0.53 mm^2 in 14nm CMOS. These results demonstrate strong cross-platform adaptability and effective hardware–software co-optimization, offering a practical matrix extension for the open-source community.

ACM Reference Format:
Anonymous Author(s). 2026. UCME: Unified and Configurable Matrix Extension for Diverse CPU Architectures with Minimal Design Overhead. In *Proceedings of 63rd ACM/IEEE Design Automation Conference (DAC '26 Under Review).* ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The rapid advancement of AI technologies has driven widespread deployment across domains such as natural language processing, computer vision, multimodal generation, and embodied intelligence, spanning platforms from edge devices to data centers[21, 26]. As a foundational component of general-purpose computing systems, CPUs continue to play a critical role in executing diverse AI tasks[4]. To address the surging demand for matrix-intensive computation, CPU vendors have integrated matrix extensions including Intel AMX[18], Arm SME[27], and IBM MMA[23] into products. The RISC-V community has also proposed several matrix extension standards such as IME[15], VME[11], and AME[29]. These extensions effectively leverage existing compute and memory resources, particularly vector units and cache subsystems, to significantly enhance AI performance with modest area and power overheads.

However, existing CPU matrix extensions pose challenges for hardware integration and software programmability[31]. Architecturally, matrix units are often tightly coupled with the CPU pipeline, entailing close interaction with vector register files or load-store units. Fine-grained synchronous matrix instructions introduce substantial structural and data hazards, increasing integration complexity and verification effort. This coupling limits portability across microarchitectures. On the software side, AI workloads typically combine matrix multiplication with a large number of element-wise operations, requiring close coordination between matrix and vector units. Expressing such fine-grained interleaving within a single instruction stream places significant burden on both programmers and compilers, complicating kernel design and scheduling.

To address these challenges, this paper proposes a unified and configurable CPU matrix extension architecture designed for agile integration and efficient execution across platforms. The matrix unit is carefully decoupled from the CPU pipeline to avoid intrusive modifications to register files and memory paths, reducing co-design and verification complexity. To accommodate diverse compute and bandwidth constraints, it supports flexible microarchitectural configurations guided by a compute-bandwidth constraint model, and can be scaled from 0.5 to 32 TOPS to guarantee resource utilization and platform adaptability. The matrix unit supports FP8/INT8/FP16/BF16/TF32 mixed-precision computing to meet varying accuracy and performance requirements. As for ISA and programming model, only an asynchronous matrix multiplication and a synchronization primitive are defined, forming a minimal and unified interface that supports flexible granularity. This abstraction hides hardware-specific details and simplifies the programming of overlapped matrix-vector execution, improving programmability and enabling a portable software stack.

We integrate and validate the proposed architecture on four open-source CPU RTL platforms: Rocket[3], Shuttle[38], BOOM[37], and XiangShan-Kunminghu[33]. We further GEMM and AI inference performance against Intel AMX, Arm SME, and IBM MMA. The matrix unit achieves over 90% utilization on GEMM workloads across all integrated platforms. On the Shuttle CPU with a 512-bit Saturn [36] vector unit, a 4 TOPS@8-bit Matrix Unit and 48 GB/s memory bandwidth, the design delivers 1.57×, 1.57×, and 2.31× speedups on ResNet[16], BERT[10], and Llama3[14] inference compared to Xeon 8580. Overlapped matrix–vector execution contributes 66.7%, 50.9%, and 33.6% of the performance gain on three workloads.The

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DAC '26 Under Review, Long Bench, CA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/XXXX/XX
https://doi.org/XXXXXXX.XXXXXXX

design also outperforms IBM S1022 MMA (8.87×, 3.33×, 3.08×) and Apple M4 SME (5.04×, 2.11×, 3.16×). A 4 TOPS@2GHz matrix unit occupies only 0.53 mm^2 in 14nm CMOS.

The key contributions of this paper are as follows:

- Proposing a unified and configurable CPU matrix extension architecture that enables agile cross-platform integration and efficient execution.
- Presenting a co-designed hardware–software solution that delivers substantial performance gains on representative AI models over commercial CPU matrix extensions.
- Integrating the proposed matrix extension into four open-source CPU RTL platforms, with all RTL implementations and high-performance kernels fully open-sourced.

2 Background

2.1 AI Workloads

AI workloads have been widely deployed across diverse computing platforms, from edge devices to data centers. Typical models consist of layers with heterogeneous characteristics. Compute-intensive layers (e.g., linear, attention, convolution) are dominated by matrix multiplications, while element-wise operations (e.g., activation, (de)quantization, normalization) are generally memory intensive.

Figure 1 illustrates the architecture and kernel fusion strategies of three representative models - ResNet, BERT, and Llama3. Kernel fusion[6, 7, 20] enhances overall performance by exploiting data locality and fusing operators into tiled pipelines, thereby reducing memory traffic and improving resource utilization.

2.2 Related Work

In recent years, CPU vendors - including Intel, Arm, and IBM - as well as the RISC-V community have introduced matrix extensions to enhance AI capabilities on general-purpose processors. These extensions typically employ fine-grained synchronous matrix instructions and differ in register architectures, execution models, and integration strategies, as illustrated in Figure 2.

IBM MMA and RISC-V IME reuse the data path of the vector unit, repurposing part of the vector register file as accumulators and restricting matrix operations to vector registers. As a result, register size and bandwidth limit the granularity and throughput of matrix operations. IBM Power10 S1022 delivers a per-core INT8 peak of 2 TOPS at 4 GHz.

Arm SME and RISC-V VME introduce dedicated accumulator registers to alleviate the capacity and bandwidth constraints of vector register files, enabling larger matrix units decoupled from vector register organization. Apple M4 implements SME on both performance and efficiency cores, with the performance core achieving a per-core INT8 peak of 4 TOPS.

Intel AMX and RISC-V AME decouple the matrix unit from the vector pipeline by introducing tile registers and dedicated load–store paths, enabling larger matrix operations. Sapphire Rapids and Emerald Rapids deliver a per-core INT8 peak of 2 TOPS/GHz, with the Xeon 8580 reaching 4.6 TOPS at 2.3 GHz under TDP limits.

The academic community has also explored CPU matrix units in both configurable forms[13, 34] and fixed-size implementations[5, 12, 17, 25]. However, these designs typically lack close cooperation with existing CPU vector units and offer limited scalability and precision support. In comparison, this work introduces an adaptable and configurable CPU matrix extension, providing the open-source community with a practical implementation.

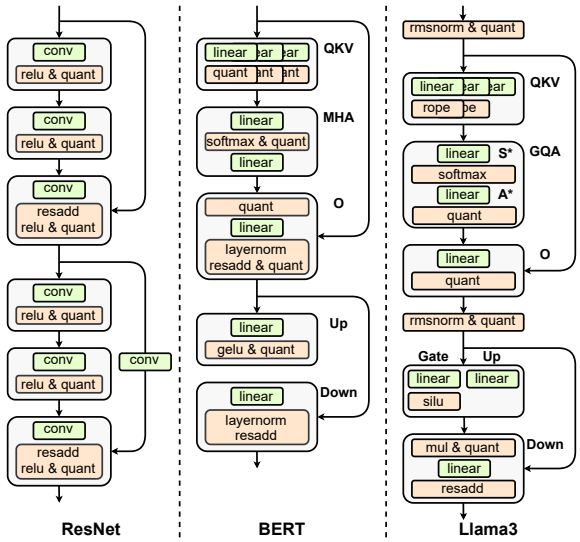

Figure 1: AI Model Architectures and Kernel Fusion Patterns.

2.3 Motivation

Although existing CPU matrix extensions have achieved significant throughput improvements, key challenges in hardware integration and software programmability still restrict their broad applicability and efficient execution across diverse CPU and workloads.

For hardware integration, most existing designs tightly couple matrix units with the CPU pipeline, necessitating invasive modifications across instruction decode, dispatch, and execution stages. Matrix units often interact with vector register files or load-store units, creating complex control paths and redundant state management, increasing integration and verification cost. In Arm SME, IBM MMA, and RISC-V IME/VME, matrix and vector instructions contend for vector registers, resulting in structural and data hazards. Although Intel AMX and RISC-V AME introduce independent tile registers and enable direct access to L1D or L2 cache, they remain constrained by synchronous semantics, which require modifications to ensure correctness between matrix and scalar/vector memory operations either by stalling potentially conflicting instructions or by resolving address conflicts within the load-store unit.

For software programmability, most CPU matrix extensions adopt fine-grained synchronous matrix instructions, forcing programmers to manually orchestrate scheduling between matrix and memory operations. Moreover, in AI workloads where matrix and element-wise kernels often require overlapped execution, programming complexity increases significantly. Developers must express memory, matrix, and vector tasks within a single instruction stream and keep all functional units busy within a limited instruction window. Furthermore, disparities among matrix-extension ISAs and software interfaces hinder unified abstractions, requiring developers to perform microarchitecture-specific optimizations and exacerbating software fragmentation.

This paper proposes a unified and configurable CPU matrix-extension architecture, designed to enable agile cross-platform integration and efficient execution with minimal design overhead.

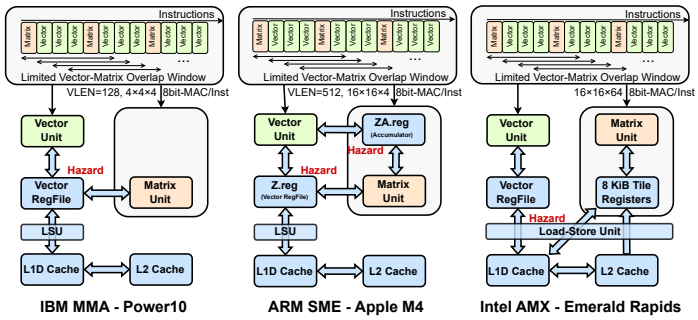

Figure 2: Existing CPU Matrix Extensions.

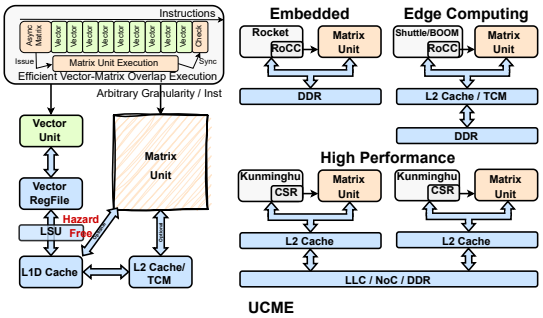

Figure 3: Architecture Overview.

3 Architecture Overview

This paper proposes a unified and configurable CPU matrix extension architecture, based on three key design principles: (1) Structural decoupling between the matrix unit and the CPU pipeline, without intrusive modifications to the decoder, instruction issue logic, register file, or load–store units; (2) Configurable microarchitectural parameters within the matrix unit to accommodate diverse computational requirements and memory system constraints across platforms; (3) An abstraction for asynchronous matrix-multiplication instructions that enables fusion scheduling with flexible granularity, which simplifies the programming model and improves execution efficiency.

Figure 3 illustrates the hardware architecture. The matrix unit is decoupled from the CPU pipeline and driven by asynchronous matrix multiplication instructions. Depending on ISA and microarchitectural support, the CPU dispatches these instructions via a RoCC-like or CSR-based interface, with registers defined in Table 1. The matrix unit connects to cache or memory independently of the CPU's load–store unit through a platform-adaptable interconnect. This design reduces integration complexity, supports rapid deployment across CPUs, and allows flexible microarchitectural configuration for embedded, edge, and high-performance platforms.

Table 1: Interface Registers.

Field	Type	Description
{M,N,K}	uint32	Matrix Size
Base {A,B,Bias,C}	uint64	Memory Base Address
Stride {A,B,Bias,C}	uint32	Memory Stride
DataType	enum	Data Precision
BiasType	enum	Bias Type (Zero, Row-Repeat, Full)
Transpose	bool	Result Transpose Flag
Status	uint32	Async Operation Status

The asynchronous matrix-multiplication abstraction substantially simplifies programming complexity for matrix extensions and enables efficient fusion of matrix–vector operations. Listing 1 shows a fused kernel example performing matrix multiplication followed by element-wise epilogue computation. The asyncMatMul macro dispatches a task per tile, with tile size determined by shared storage capacity between CPU and matrix unit. During asynchronous execution, the CPU issue window can be fully utilized by the vector unit to compute epilogue operations. Before vector computation proceeds, the checkMatmul instruction ensures the matrix multiplication of the corresponding tile is complete, thereby handling data dependencies correctly.

```
asyncMatMul(TILE_M, TILE_N, K, ...);      // tile 0
for (i=1; i<(M/TILE_M)*(N/TILE_N); i++) {
    asyncMatMul(TILE_M, TILE_N, K, ...); // tile i
    checkMatmul(); // wait tile i-1
    ...            // tile i-1 epilogue
}
checkMatmul();     // wait last tile
...                // last tile epilogue
```

Listing 1: Programming Example.

4 Design

4.1 Matrix Unit Microarchitecture

As illustrated in Figure 4, the matrix unit consists primarily of the Memory Loader, Scratchpad, Data Controller, and PE Array.

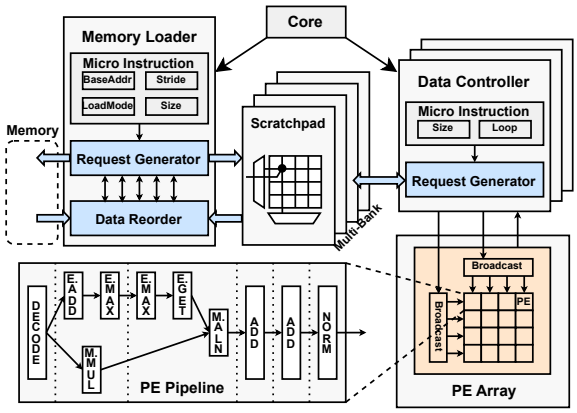

Figure 4: Matrix Unit Microarchitecture.

Memory Loader generates memory access requests and handles all data reads and writes for matrix computations. Within it, the Request Generator translates the tensor described by matrix multiplication instructions into memory and Scratchpad addresses, generating and issuing the corresponding requests. The Data Reorder module receives the returned data and reorder it as required, and writes it to the target storage.

Scratchpad temporarily stores matrix partitions to improve data reuse in the matrix unit. Accumulation results can remain resident in the Scratchpad, thereby reducing the need to write back high-precision results. A multi-bank Scratchpad further enables overlapping of data loading and computation tasks.

Data Controller supplies data to the computation array. Three Data Controllers are instantiated: two dedicated to source matrices A and B in matrix multiplication, and one dedicated to bias and accumulation matrix C.

PE Array combines outer-product and vector dot-product operations, where operands A and B are broadcast row-wise and column-wise across the array, respectively. Each PE performs inner-product operations and supports mixed-precision computing for TF32, BF16, FP16, INT8, and FP8 formats. Within each PE, multiplication results are aligned to a common exponent, truncated, and accumulated. The PE is organized into a six-stage pipeline to achieve a 2 GHz operating frequency with 14nm process node.

4.2 Configurable Matrix Extension

Table 2: Configurable Architectural Parameters.

Parameters	Meaning	Case Study
$Freq$	Clock Frequency	2.0 GHz
M_{pe}	Row of PE Array	4
N_{pe}	Column of PE Array	4
K_{pe}	PE Reduce Width	512 Bits
M_{scp}	Max Resident M in Scratchpad	64
N_{scp}	Max Resident N in Scratchpad	64
K_{scp}	Max Resident K in Scratchpad	64 Bytes
	Data Bandwidth	48 GB/s
	Throughput (8-bit)	4 TOPS

In order to support the generation of implementations with different compute capabilities, we provide a set of configurable microarchitectural parameters to tune the matrix unit for the compute requirements of diverse SoCs.

Table 2 lists the configurable microarchitectural parameters of the matrix unit, with a case-study configuration aligned with the compute throughput and data bandwidth of Intel Xeon 8580 AMX.

The throughput of the PE array is determined by M_{pe}, N_{pe}, and K_{pe}. For an n-bit data format, the theoretical throughput is:

$$\text{Throughput (n-bit)} = Freq \times M_{pe} \times N_{pe} \times (K_{pe}/n) \times 2 \quad (1)$$

The parameters M_{scp}, N_{scp}, and K_{scp} define the scratchpad size. By adjusting the scratchpad capacity, the data-reuse level of the matrix unit can be adjusted to match different memory-bandwidth constraints. To avoid wasted compute, it is necessary to ensure that, under the output-stationary scheduling strategy, the compute time in the matrix-multiplication loop does not exceed the memory-access time:

$$\frac{M_{scp} \times N_{scp} \times K_{scp}}{Freq \times M_{pe} \times N_{pe} \times K_{pe}} \leq \frac{(M_{scp} + N_{scp}) \times K_{scp}}{DataBandwidth} \quad (2)$$

Here, $DataBandwidth$ denotes the data-supply bandwidth of the lower-level memory hierarchy, determined by the cache structure, on-chip network and QoS, memory bandwidth, and other related system factors.

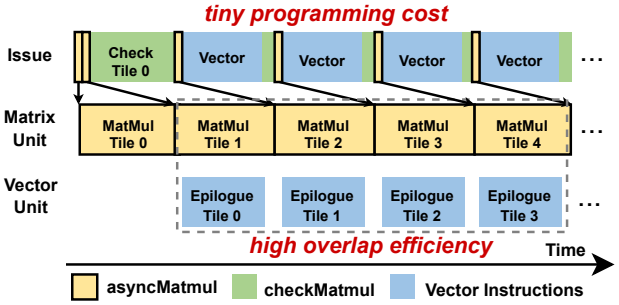

Figure 5: Vector and Matrix overlap.

4.3 Vector-Matrix Overlap

The asynchronous matrix multiplication abstraction allows matrix tasks to be issued from the CPU pipeline without occupying it until completion, creating more opportunities for matrix–vector overlap. For the operators shown in Figure 1, we adopt the programming method illustrated in Listing 1 to implement matrix–vector fused kernels, with the vector unit executing the prologue and epilogue while the matrix unit handling linear and convolution. Matrix multiplication and vector operations are orchestrated in a software pipeline at the granularity of matrix tiling, producing the fused execution behavior depicted in Figure 5.

4.4 Low Design Overhead Integration

The proposed matrix extension was integrated into the four open-source CPU RTL platforms listed in Table 3, covering architectures from in-order single-issue to out-of-order six-issue designs. For Rocket, Shuttle, and BOOM, the integration reused the RoCC[3] interface and was completed in a few days; for the XiangShan-Kunminghu processor, a new CSR interface was added, taking several weeks. Across all four processors, integrating the matrix extension required only 200–500 additional lines of RTL code, demonstrating the ease of integration of the proposed design.

Table 3: Development Cost of Integration.

CPU	Micro Architecture	Interface	Code*	Time
Rocket[3]	In-order, 1-issue	RoCC	254	3 days
Shuttle[38]	In-order, 3-issue	RoCC	512	5 days
BOOM[37]	Out-of-order, 4-issue	RoCC	301	3 days
Xiangshan[33]	Out-of-order, 6-issue	CSR	361	3 weeks

*Code indicates the lines of integration-related RTL modifications.

5 Evaluation

5.1 Methodology

The experimental setup is shown in Table 4, with Rocket, Shuttle, and BOOM evaluated on the Chipyard platform, and XiangShan-Kunminghu on the XiangShan platform. Table 4 also lists parameter configurations used to validate the scalability of the matrix extension. For reference, matrix extension configurations of three commercial CPUs are shown in the case-study column of Table 2.

This work is compared against three commercial matrix extensions representative of prior work, as listed in Section 2.2. All baselines are evaluated with 8-bit inference on ResNet-50 v1.5, BERT-base, and Llama3.2-1B, with Llama3.2-1B quantized using

Table 4: Experimental Setup and Configuration.

Parameter	Configuration
SoC Framework	Chipyard[1] + Verilator + DRAMSim[28]
CPU Core	Rocket / Shuttle / BOOM / Kunminghu*
Vector Unit	512-bit RVV Saturn [36]
PE Array Size	2×2 / 4×4 / 8×8 / 16×16
PE Reduce Width	256 / 512 Bits
Clock Frequency	2.0 GHz
Data Bandwidth	8 / 16 / 32 / 48 / 64 GB/s

*Kunminghu is evaluated on XiangShan[33] SoC platform.

Table 5: Baseline configurations

Platform	Framework	ISE	Bandwidth	INT8 Peak
IBM S1022	ONNX Runtime[9]	MMA	52.37 GB/s	2 TOPS
Apple M4	ONNX Runtime[9]	SME	131.31 GB/s	4 TOPS
Xeon 8580	OpenVINO[8]	AMX	49.48 GB/s	4.6 TOPS

SmoothQuant-O1[32] to maintain accuracy. Peak performance and per-core memory bandwidth are reported in Table 5, with memory bandwidth measured using MLC[19] and STREAM[22]. IBM S1022 is evaluated with two threads to fully utilize the MMA unit[30]. Intel AMX is tested with OneDNN v3.9[24] and OpenVINO 2025.2.0[8]; Arm SME with KleidiAI 1.14.0[2] and ONNX Runtime (ORT) 1.21[9]; IBM MMA with OpenBLAS 0.3.27[35] and ORT 1.16.3.

5.2 Integration Across Various CPU Platforms

The matrix extensions on four CPU platforms including Rocket, Shuttle, BOOM, and Kunminghu are configured for a peak throughput of 2 TOPS, and GEMM workloads are evaluated with $M = 512$, $N = 512$, and K ranging from 256 to 8192. As shown in Figure 6, all platforms achieve matrix unit utilization above 90%. The results demonstrate the adaptability of our matrix extension across different CPU architectures.

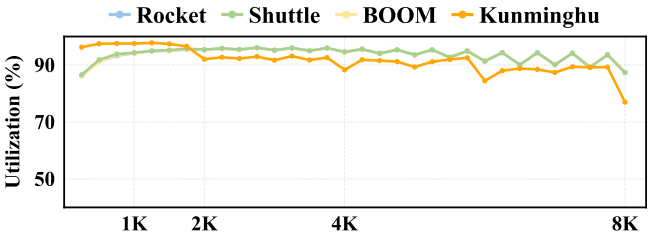

Figure 6: GEMM Performance on Various CPU Platforms.

5.3 Matrix Unit Configurations at Various Scales

Matrix extension configurations are instantiated on the Shuttle CPU under four bandwidth settings, each targeting distinct peak compute capabilities. As shown in Figure 7, scratchpad sizes are configured according to Equation 2. The GEMM workload follows Section 5.2. Matrix unit utilization reaches approximately 80% across all configurations. The proposed matrix extension adapts to both bandwidth-constrained embedded scenarios and high-performance CPUs, demonstrating strong scalability.

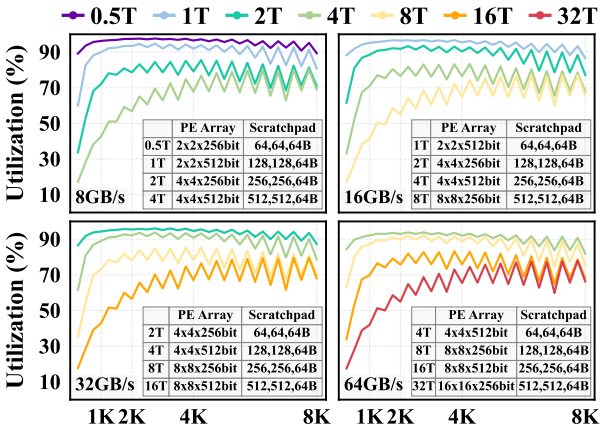

Figure 7: GEMM Performance under Different Config.

5.4 Performance under Various Workloads

In this section, the matrix extension on Shuttle CPU is configured as in the case study of Table 2 and paired with a 512-bit Saturn[36] vector unit, to serve as a counterpart to the Xeon 8580 AMX extension. It is evaluated against the three commercial processors from Table 5 on four representative workloads.

For **GEMM**, as shown in Figure 8, the proposed matrix extension outperforms the Xeon 8580 (OneDNN) and IBM S1022 (OpenBLAS), and approaches Apple M4 (KleidiAI). Performance fluctuations are caused by variations in DRAMSim memory throughput under different stride access patterns.

For **ResNet-50**, as shown in Figure 9, AMX (OpenVINO) provides better operator support compared with MMA (ORT) and SME (ORT). Benefiting from 300 MiB L3 cache on Xeon 8580, all weights and activations can be buffered, thus enabling higher effective memory bandwidth. As for now, SME lacks support for convolution operators. Without vector-matrix fused kernels, the performance of our implementation is comparable to AMX. With fused kernels enabled, overall performance achieves speedups of 1.57×, 8.87×, and 5.04× compared to Xeon 8580, IBM S1022, and Apple M4, respectively.

For **BERT**, as shown in Figure 10, Transformer models dominated by small-scale matrix multiplications pose challenges for matrix unit utilization. Vendor extensions only achieve moderate utilization limited by small matrix multiplication sizes. With superior support for small-scale matrices, our implementation achieves speedups of 1.57×, 3.33×, and 2.11× compared to Xeon 8580 (OpenVINO), IBM S1022 (ORT), and Apple M4 (ORT), respectively.

For **Llama3**, as shown in Figure 11, the Llama3 model has a more complex Transformer structure than BERT and requires newer quantization methods such as SmoothQuant[32] to control accuracy loss, which necessitates agile hardware–software co-optimization. Compared to BERT, Llama3 involves larger-scale matrix multiplications, enabling our matrix extension to achieve higher utilization. This also makes the execution time of vector operations comparable to matrix operations, yielding significant benefits from kernel fusion. Our matrix extension achieves speedups of 2.31×, 3.08×, and 3.16× compared to Xeon 8580 (OpenVINO), IBM S1022 (ORT), and Apple M4 (ORT), exceeding those observed for BERT. This result indicates that there is substantial room for hardware–software co-optimization of CPU matrix extensions on emerging workloads.

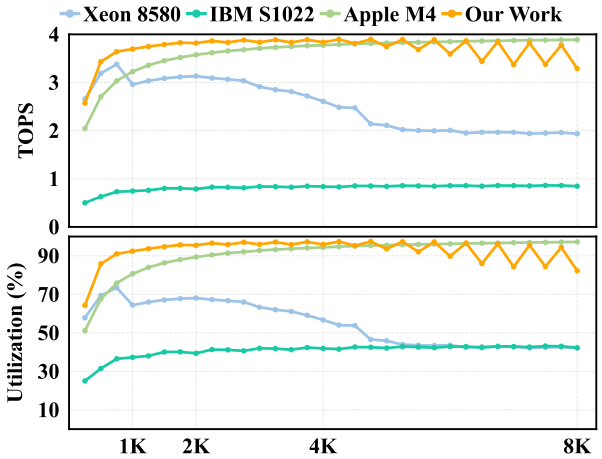

Figure 8: GEMM Performance vs. Existing Extensions.

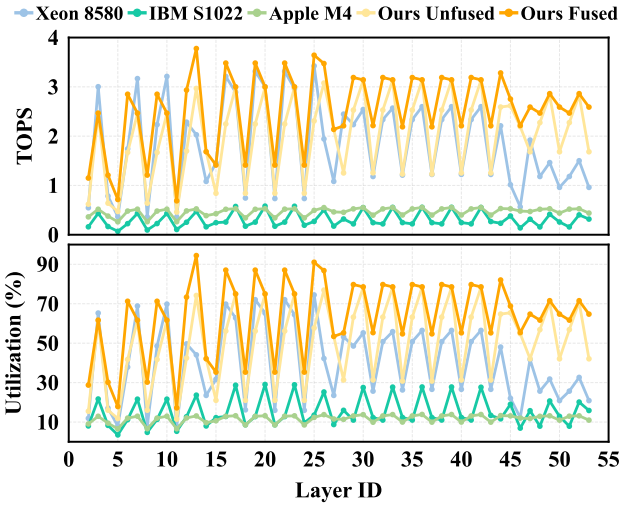

Figure 9: ResNet-50 Performance vs. Existing Extensions.

Our implementation exhibits relatively low performance on the Score (S*) operation due to Softmax dominance and limited vector throughput on Saturn. The significant gap between Gate and Up is due to the floating-point division in SiLU. On Saturn, vector division is implemented element-wise and is constrained by vector throughput. This highlights the optimization potential of current open-source RVV vector units.

The speedups of our matrix extension over three commercial processor extensions are shown in Table 6. Current kernel implementations on commercial platforms do not fully leverage the matrix hardware. The substantial improvements observed in our fused implementation compared to unfused kernels suggest that the proposed programming model makes it easier to harness the full potential of matrix hardware.

Table 6: Speed Up.

	Xeon 8580			IBM S1022			Apple M4		
	R	B	L	R	B	L	R	B	L
Ours unfused	1.19	1.28	1.87	7.16	2.72	2.39	3.82	1.72	2.55
Ours fused	1.57	1.57	2.31	8.87	3.33	3.08	5.04	2.11	3.16

R, B, and L denote ResNet-50, BERT-base, and Llama3.2-1B, respectively.

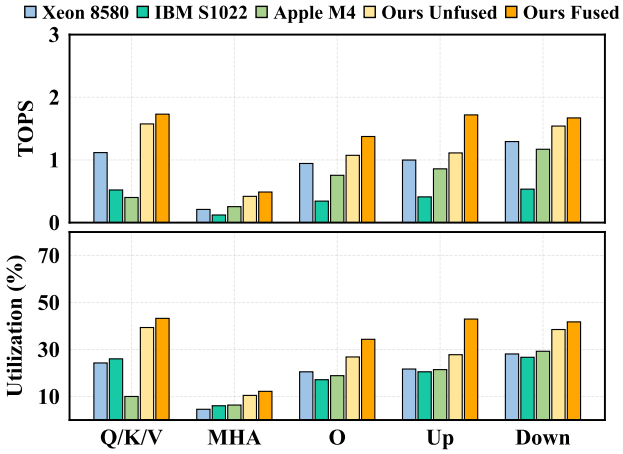

Figure 10: BERT Performance vs. Existing Extensions.

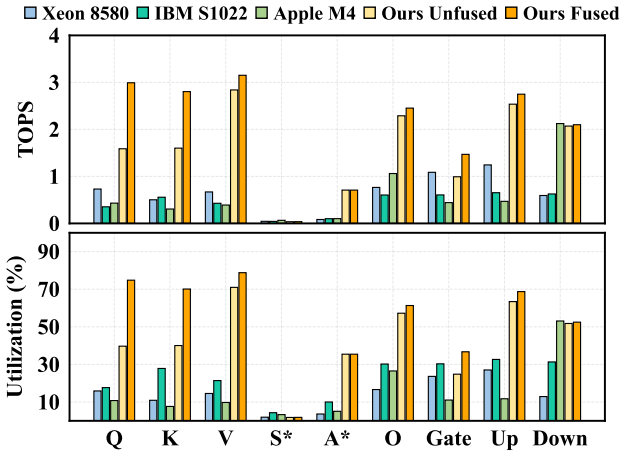

Figure 11: Llama3 Performance vs. Existing Extensions.

5.5 Area and Power

The matrix extension configurations from Section 5.4 were synthesized in a 14nm process technology. The operating frequency reaches 2 GHz, and Table 7 reports the area and power consumption.

Table 7: Area and Power Evaluation (4TOPS@2GHz).

	Area (mm^2)	Power (W)
RAM	0.164	0.784
Logic	0.367	0.722
Total	0.531	1.506

6 Conclusion

Overall, this paper presents a unified and configurable CPU matrix extension architecture that facilitates agile cross-platform integration and efficient execution. The architecture has been deployed across multiple open-source CPU platforms, consistently delivering high performance under diverse configurations and workloads, thereby confirming its adaptability and scalability. This contribution offers the open-source community a practical solution for CPU matrix extensions. Future work will continue to explore the potential of CPU matrix extensions.

References

[1] Alon Amid, David Biancolin, Abraham Gonzalez, Daniel Grubb, Sagar Karandikar, Harrison Liew, Albert Magyar, Howard Mao, Albert Ou, Nathan Pemberton, Paul Rigge, Colin Schmidt, John Wright, Jerry Zhao, Yakun Sophia Shao, Krste Asanović, and Borivoje Nikolić. 2020. Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs. *IEEE Micro* 40, 4 (2020), 10–21.

[2] Arm. 2025. KleidiAI. https://gitlab.arm.com/kleidi/kleidiai.

[3] Krste Asanović, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, Sagar Karandikar, Ben Keller, Donggyu Kim, John Koenig, Yunsup Lee, Eric Love, Martin Maas, Albert Magyar, Howard Mao, Miquel Moreto, Albert Ou, David A. Patterson, Brian Richards, Colin Schmidt, Stephen Twigg, Huy Vo, and Andrew Waterman. 2016. *The Rocket Chip Generator.* Technical Report. http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html

[4] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)* (Lotte Hotel World, Seoul, Republic of Korea). ACM, 1014–1029.

[5] Francesco Conti, Gianna Paulin, Angelo Garofalo, Davide Rossi, Alfio Di Mauro, Georg Rutishauser, Gianmarco Ottavi, Manuel Eggiman, Hayate Okuhara, and Luca Benini. 2024. Marsellus: A Heterogeneous RISC-V AI-IoT End-Node SoC With 2–8 b DNN Acceleration and 30%-Boost Adaptive Body Biasing. *IEEE Journal of Solid-State Circuits* 59, 1 (2024), 128–142.

[6] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).

[7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems(NIPS)* (New Orleans, LA, USA). Curran Associates Inc., Article 1189, 16 pages.

[8] OpenVINO developers. 2025. Intel Distribution of OpenVINO Toolkit. https://github.com/openvinotoolkit/openvino.

[9] ONNX Runtime developers. 2021. ONNX Runtime. https://onnxruntime.ai/.

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)* (Minneapolis, MN, USA). Association for Computational Linguistics, 4171–4186.

[11] Greg Favor. 2025. Vector-Matrix Extension. https://riscv.atlassian.net/wiki/spaces/VMEX/pages/554991628/Vector-Matrix+Extension+VME+-+PoW.

[12] Peng Gao, Yang Liu, Haonan Sun, Jiang Jiang, Jun Wang, Zonghui Hong, and Jiali Qu. 2025. OASIS: A Commercial High Performance Terminal AI Processor Supporting RISC-V Tensor Extension Instructions. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO)* (New York, NY, USA). ACM, 1264–1283.

[13] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, Albert Ou, Colin Schmidt, Samuel Steffl, John Wright, Ion Stoica, Jonathan Ragan-Kelley, Krste Asanovic, Borivoje Nikolic, and Yakun Sophia Shao. 2021. Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration. In *2021 58th ACM/IEEE Design Automation Conference (DAC)* (San Francisco, CA, USA). 769–774.

[14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models.

[15] Rafael Sene Erich Focht Guido Araujo, Jose Moreira. 2025. Integrated Matrix Extension. https://github.com/riscv-admin/integrated-matrix-extension.

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (Las Vegas, NV, USA). IEEE, 770–778.

[17] Pouya Houshmand, Giuseppe M. Sarda, Vikram Jain, Kodai Ueyoshi, Ioannis A. Papistas, Man Shi, Qilin Zheng, Debjyoti Bhattacharjee, Arindam Mallik, Peter Debacker, Diederik Verkest, and Marian Verhelst. 2023. DIANA: An End-to-End Hybrid DIgital and ANAlog Neural Network SoC for the Edge. *IEEE Journal of Solid-State Circuits* 58, 1 (2023), 203–215.

[18] Intel. 2024. Intel 64 and IA-32 Architectures Optimization Reference Manual. https://cdrdv2-public.intel.com/814201/355308-Optimization-Reference-Manual-049-Changes-Doc.pdf.

[19] Intel. 2025. Intel Memory Latency Checker v3.12. https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html.

[20] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, Daniel Haziza, Luca Wehrstedt, Jeremy Reizenstein, and Grigory Sizov. 2022. xFormers: A modular and hackable Transformer modelling library. https://github.com/facebookresearch/xformers.

[21] Héctor Martínez, Adrián Castelló, Francisco D. Igual, and Enrique S. Quintana-Ortí. 2026. The cambrian explosion of mixed-precision matrix multiplication for quantized deep learning inference. *Future Generation Computer Systems* (2026).

[22] John D. McCalpin. 1995. Memory Bandwidth and Machine Balance in Current High Performance Computers. *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter* (Dec. 1995).

[23] José E Moreira, Kit Barton, Steven Battle, Peter Bergner, Ramon Bertran, Puneeth Bhat, Pedro Caldeira, David Edelsohn, Gordon Fossum, Brad Frey, et al. 2021. A matrix math facility for Power ISA (TM) processors. *arXiv preprint arXiv:2104.03142* (2021).

[24] oneDNN Contributor. 2025. oneAPI Deep Neural Network Library (oneDNN). https://github.com/uxlfoundation/oneDNN.

[25] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)* (San Diego, CA, USA). 58–70.

[26] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf inference benchmark. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture(ISCA)* (Virtual Event). IEEE Press, 446–459.

[27] Stefan Remke and Alexander Breuer. 2024. Hello SME! Generating Fast Matrix Multiplication Kernels Using the Scalable Matrix Extension. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis(SC24-W)* (Atlanta, GA, USA). IEEE Press, 1443–1454.

[28] Paul Rosenfeld, Elliott Cooper-Balis, and Bruce Jacob. 2011. DRAMSim2: A Cycle Accurate Memory System Simulator. *IEEE Computer Architecture Letters* 10, 1 (2011), 16–19.

[29] Rafael Sene and Philipp Tomsich. 2023. Attached Matrix Extension. https://github.com/riscv-admin/attached-matrix-extension.

[30] William J. Starke and Brian W. Thompto. 2020. IBM's POWER10 Processor. In *IEEE Hot Chips 32 Symposium, HCS 2020, Palo Alto, CA, USA, August 16-18, 2020* (Palo Alto, CA, USA). IEEE, 1–43.

[31] Josse Van Delm, Anton Lydike, Joren Dumoulin, Jonas Crols, Xiaoling Yi, Ryan Antonio, Jackson Woodruff, Tobias Grosser, and Marian Verhelst. 2025. The Configuration Wall: Characterization and Elimination of Accelerator Configuration Overhead. To appear in Proceedings of ASPLOS 2026.

[32] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *International Conference on Machine Learning(ICML)* (Honolulu, Hawaii, USA) *(Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 38087–38099.

[33] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, Jiawei Lin, Tong Liu, Zhigang Liu, Jiazhan Tan, Huaqiang Wang, Huizhe Wang, Kaifan Wang, Chuanqi Zhang, Fawang Zhang, Linjuan Zhang, Zifei Zhang, Yangyang Zhao, Yaoyang Zhou, Yike Zhou, Jiangrui Zou, Ye Cai, Dandan Huan, Zusong Li, Jiye Zhao, Zihao Chen, Wei He, Qiyuan Quan, Xingwu Liu, Sa Wang, Kan Shi, Ninghui Sun, and Yungang Bao. 2023. Towards Developing High Performance RISC-V Processors Using Agile Methodology. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture(MICRO)* (Chicago, Illinois, USA). IEEE Press, 1178–1199.

[34] Xiaoling Yi, Ryan Antonio, Joren Dumoulin, Jiacong Sun, Josse Van Delm, Guilherme Pereira Paim, and Marian Verhelst. 2025. OpenGeMM: A Highly-Efficient GeMM Accelerator Generator with Lightweight RISC-V Control and Tight Memory Coupling. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference* (Tokyo, Japan). ACM, 1055–1061.

[35] Martin Kroeker Zhang Xianyi. 2025. OpenBLAS. https://github.com/OpenMathLib/OpenBLAS.

[36] Jerry Zhao, Daniel Grubb, Miles Rusch, Tianrui Wei, Kevin Anderson, Borivoje Nikolic, and Krste Asanović. 2024. *The Saturn Microarchitecture Manual.* Technical Report. EECS Department, University of California, Berkeley. http://www2.eecs.berkeley.edu/Pubs/TechRpts/2024/EECS-2024-215.html

[37] Jerry Zhao, Ben Korpan, Abraham Gonzalez, and Krste Asanovic. 2020. SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine. In *Fourth Workshop on Computer Architecture Research with RISC-V (CARRV).*

[38] Jerry Zhao, Jennifer Zhou, Albert Ou, Abraham Gonzalez, and Lux Zhang. 2023. Shuttle: A Rocket-based Superscalar In-order RISC-V Core. https://github.com/ucb-bar/shuttle/tree/master.